Aprendizaje Profundo

Alvaro J. Riascos Villegas

Uniandes y Quantil

Noviembre de 2026

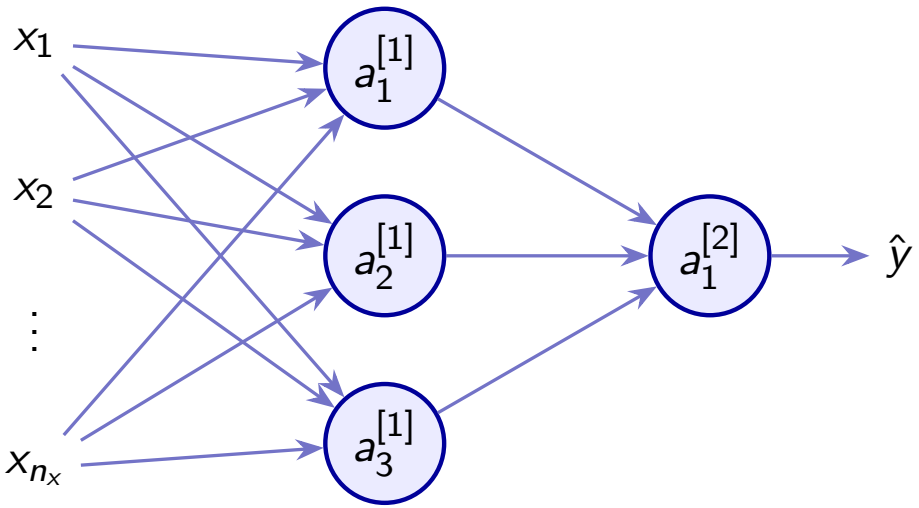
Contenido

1. Redes con dos capas
2. Redes profundas
3. Funciones de activación
4. Aproximación de funciones
5. Salida multiclase

Notación

- L: número de capas (sin contar la entrada); las capas se indexan con $l = 1, \ldots, L$.
- $n^{[l]}$: número de neuronas de la capa l; $n_x = n^{[0]}$ es el número de variables de entrada y $n_y = n^{[L]}$ el tamaño de la salida (o número de clases).
- Entrada y salida: $a^{[0]} = x \in \mathbb{R}^{n_x}$ y $\hat{y} = a^{[L]}$.
- Parámetros de la capa l: $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$ (elementos $w_{jk}^{[l]}$) y $b^{[l]} \in \mathbb{R}^{n^{[l]}}$.
- Índices: el superíndice $[l]$ indica la capa, el superíndice (i) el ejemplo y el subíndice j la neurona; por ejemplo, $a_j^{[l](i)}$.

Arquitectura de una red con dos capas



Entrada
$x \in \mathbb{R}^{n_x}$

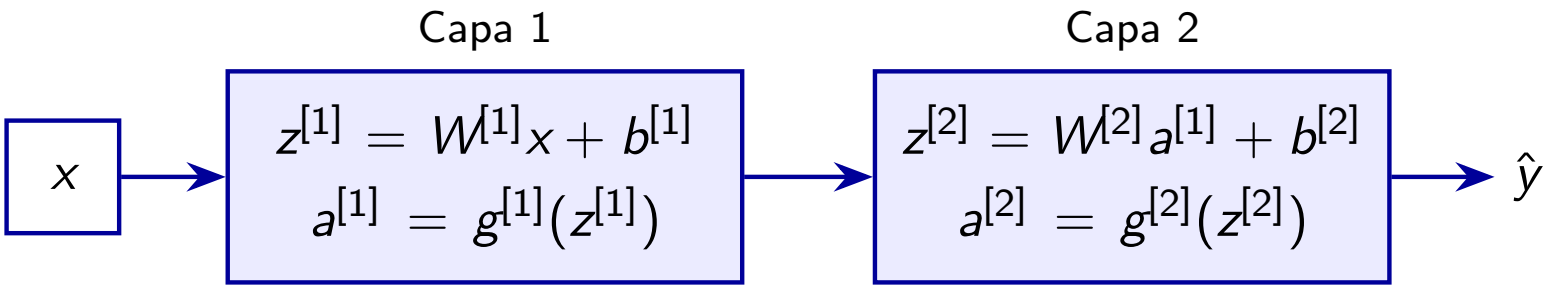
Capa oculta
$l = 1$

Capa de salida
$l = 2$

$$z^{[1]} = W^{[1]}x + b^{[1]},$$
$$a^{[1]} = g^{[1]}(z^{[1]}),$$
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]},$$
$$\hat{y} = a^{[2]} = \sigma(z^{[2]}).$$

Aquí $n^{[1]} = 3$ y $n^{[2]} = 1$:
$W^{[1]} \in \mathbb{R}^{3 \times n_x}$, $W^{[2]} \in \mathbb{R}^{1 \times 3}$.

Propagación hacia adelante



- Cada capa recibe la activación de la capa anterior; la salida es $\hat{y} = a^{[2]}$.
- Con m ejemplos se apilan en columnas, $X = [x^{(1)} \dots x^{(m)}] \in \mathbb{R}^{n_x \times m}$, y se calcula todo a la vez: $Z^{[1]} = W^{[1]}X + b^{[1]}$, $A^{[1]} = g^{[1]}(Z^{[1]})$, etc.

Contenido

1 Redes con dos capas

2 Redes profundas

3 Funciones de activación

4 Aproximación de funciones

5 Salida multiclase

Propagación hacia adelante: caso general

Propagación hacia adelante

Con $a^{[0]} = x$, para $l = 1, \ldots, L$:

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]},$$
$$a^{[l]} = g^{[l]}(z^{[l]}),$$

y el pronóstico es $\hat{y} = a^{[L]}$.

Retropropagación

- El gradiente con respecto a $W^{[2]}, b^{[2]}$ es idéntico al de la regresión logística (clase anterior), excepto que ahora $a^{[1]}$ juega el papel de x.
- Las siguientes diapositivas explican de forma esquemática el álgebra detrás del cálculo del gradiente. La forma eficiente de calcularlo es lo que se conoce como **retropropagación** (*backpropagation*) (Rumelhart et al., 1986).
- Este algoritmo se generaliza de forma natural al caso de redes profundas (es decir, redes con muchas capas ocultas).
- Notación para un ejemplo:

$$dz^{[l]} \equiv \frac{\partial \mathcal{L}}{\partial z^{[l]}}, \qquad dW^{[l]} \equiv \frac{\partial \mathcal{L}}{\partial W^{[l]}}, \qquad db^{[l]} \equiv \frac{\partial \mathcal{L}}{\partial b^{[l]}}.$$

Retropropagación: capa de salida

Capa L (salida sigmoide y pérdida de entropía cruzada)

$$dz^{[L]} = a^{[L]} - y,$$
$$dW^{[L]} = dz^{[L]}\, a^{[L-1]\top},$$
$$db^{[L]} = dz^{[L]}.$$

- Es el mismo resultado de la regresión logística, con $a^{[L-1]}$ en lugar de x.

Retropropagación: capas ocultas

Capa $l < L$

$$
\begin{aligned}
dz^{[l]} &= \left(W^{[l+1]\top} dz^{[l+1]}\right) \odot g^{[l]\prime}(z^{[l]}), \\
dW^{[l]} &= dz^{[l]}\, a^{[l-1]\top}, \\
db^{[l]} &= dz^{[l]}.
\end{aligned}
$$

- $\odot$ denota el producto elemento a elemento.
- El error dz se propaga de la capa $l+1$ a la capa l: de ahí el nombre *retropropagación*.

Retropropagación: varios ejemplos

- Se apilan los m ejemplos en columnas: $A^{[l]} = [a^{[l](1)} \dots a^{[l](m)}] \in \mathbb{R}^{n^{[l]} \times m}$, $Y = [y^{(1)} \dots y^{(m)}]$, y $dZ^{[l]} = [dz^{[l](1)} \dots dz^{[l](m)}]$.

Gradiente de $J = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$

$$dZ^{[L]} = A^{[L]} - Y,$$

$$dZ^{[l]} = \left(W^{[l+1]\top} dZ^{[l+1]}\right) \odot g^{[l]\prime}(Z^{[l]}), \quad l < L,$$

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} A^{[l-1]\top}, \qquad db^{[l]} = \frac{1}{m} \sum_{i=1}^{m} dz^{[l](i)}.$$

- Actualización: $W^{[l]} \leftarrow W^{[l]} - \alpha\, dW^{[l]}$, $\quad b^{[l]} \leftarrow b^{[l]} - \alpha\, db^{[l]}$.

Contenido

1 Redes con dos capas

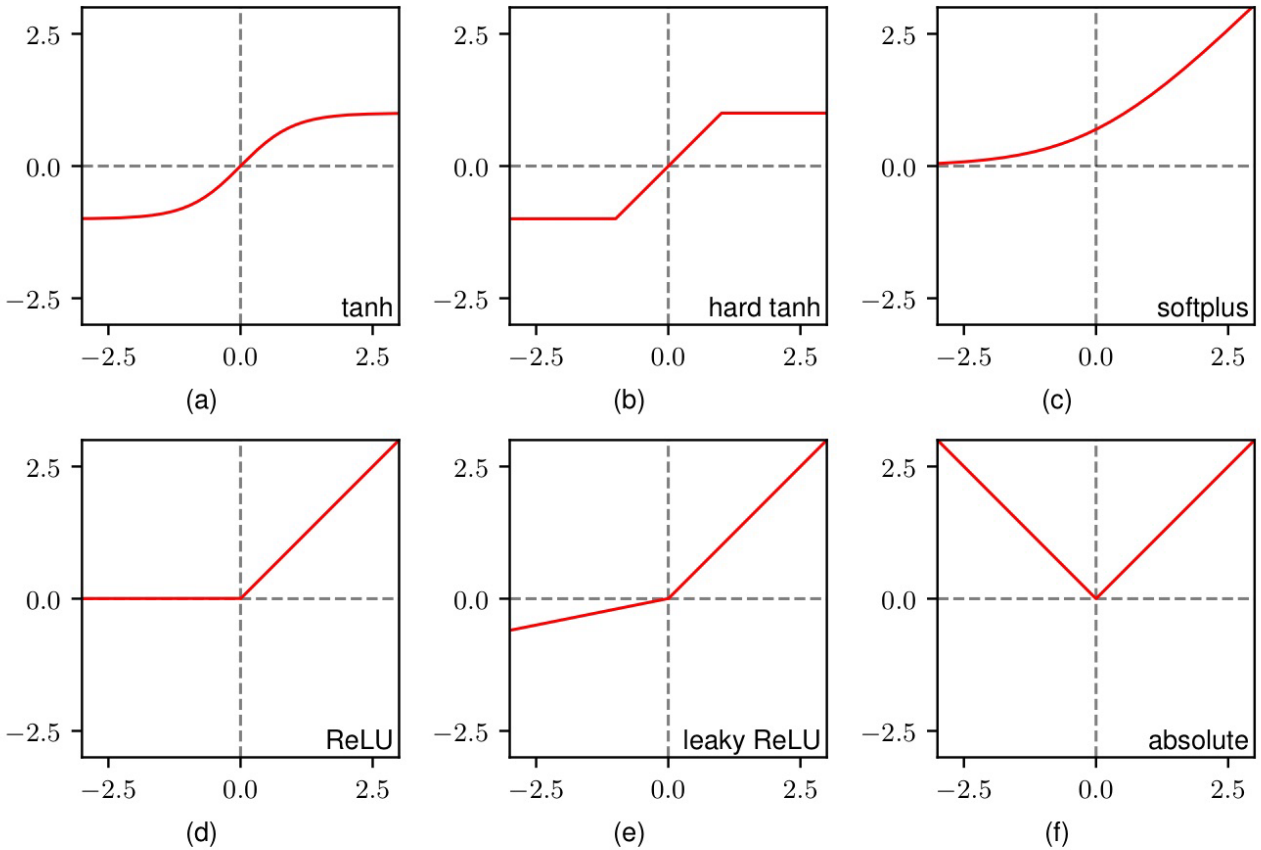
2 Redes profundas

3 Funciones de activación

4 Aproximación de funciones

5 Salida multiclase

Funciones de activación



Fuente: Bishop and Bishop (2024), fig. 6.12.

Funciones de activación: definiciones

Sigmoide	$\sigma(z) = \dfrac{1}{1+e^{-z}}$
(a) Tangente hiperbólica	$\tanh(z) = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$
(b) *Hard* tanh	$\max\{-1, \min\{1, z\}\}$
(c) *Softplus*	$\ln(1+e^{z})$
(d) ReLU	$\max\{0, z\}$
(e) *Leaky* ReLU	$\max\{\alpha z, z\}, \quad 0 < \alpha < 1$
(f) Valor absoluto	$\lvert z \rvert$

- Sin activaciones no lineales, la composición de capas es una función lineal: la red equivale a un modelo lineal.

Contenido

1 Redes con dos capas

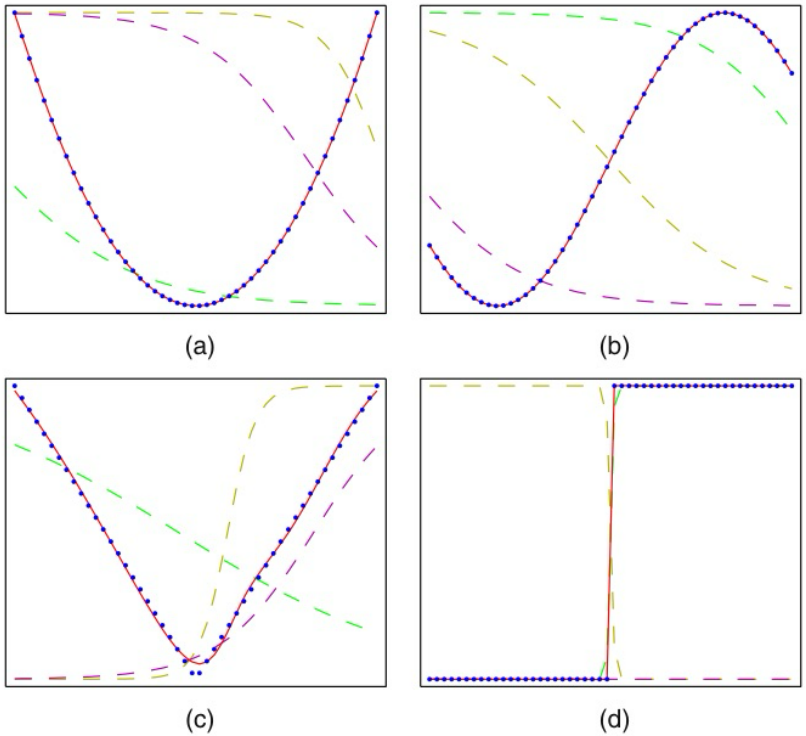
2 Redes profundas

3 Funciones de activación

4 Aproximación de funciones

5 Salida multiclase

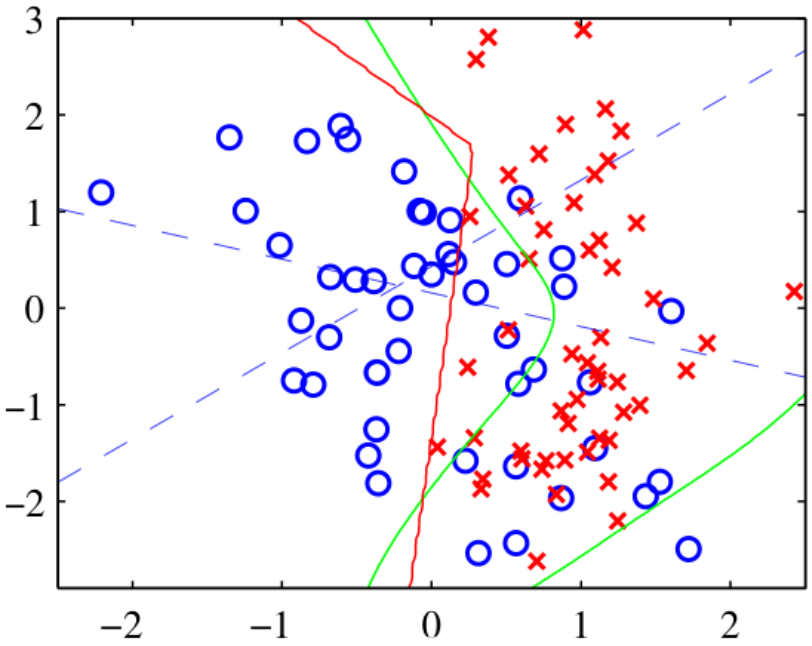
Aproximación de funciones



Fuente: Bishop (2006), fig. 5.3.

- Red con dos capas: tres neuronas ocultas con activación tanh y salida lineal, entrenada con 50 puntos (azules).
- Funciones aproximadas: (a) cuadrática, (b) seno, (c) valor absoluto y (d) función de Heaviside.
- Líneas punteadas: salidas de las tres neuronas ocultas. Línea roja: salida de la red.

Ejemplo: clasificación



Fuente: Bishop (2006), fig. 5.4.

- Red con dos neuronas ocultas (tanh) y salida sigmoide logística.
- Líneas azules punteadas: salida de cada neurona oculta.
- Línea roja: frontera de decisión de la red ($\hat{y} = 0{,}5$).
- Línea verde: frontera bayesiana óptima (con las distribuciones que generaron los datos).

Contenido

1 Redes con dos capas

2 Redes profundas

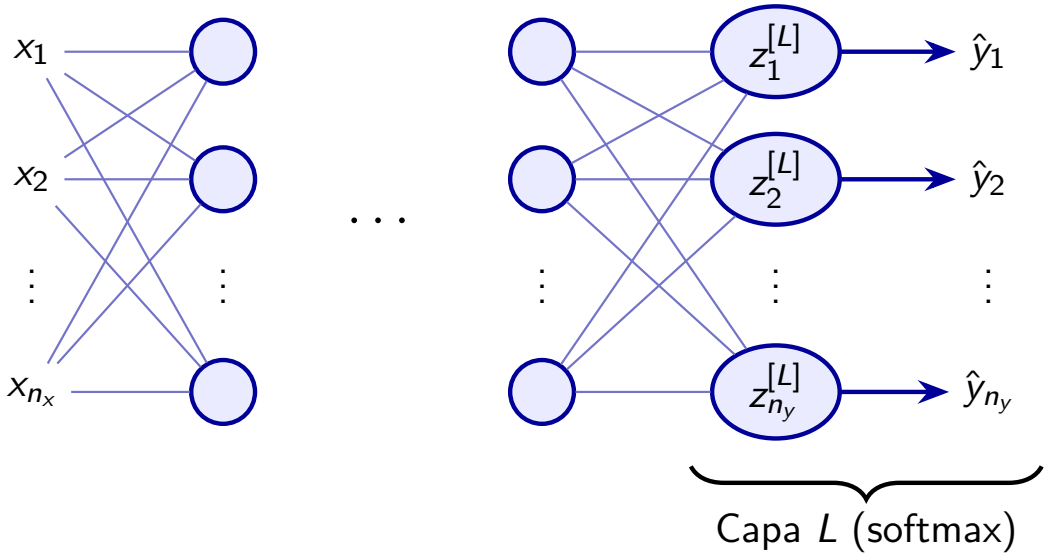
3 Funciones de activación

4 Aproximación de funciones

5 Salida multiclase

Salida multiclase: softmax

$$\hat{y}_k = P(y = k \mid x) = \frac{e^{z_k^{[L]}}}{\sum_{j=1}^{n_y} e^{z_j^{[L]}}}, \qquad k = 1, \ldots, n_y.$$



- La última capa es especial: softmax es una activación vectorial que acopla las n_y salidas, de modo que $\hat{y}_k > 0$ y $\sum_k \hat{y}_k = 1$.

Salida multiclase: función de pérdida

- La etiqueta se codifica como vector *one-hot*: $y \in \{0,1\}^{n_y}$, con $y_k = 1$ si el ejemplo pertenece a la clase k.

Entropía cruzada

$$\mathcal{L}(\hat{y}, y) = -\sum_{k=1}^{n_y} y_k \ln \hat{y}_k.$$

Con $n_y = 2$ se recupera la pérdida de la regresión logística.

Gradiente en la capa de salida

$$dz^{[L]} = \hat{y} - y.$$

Es la misma forma que en el caso binario; la retropropagación hacia las capas ocultas no cambia.

Retropropagación con softmax: capa de salida

- Como $\hat{y}_k$ depende de todas las $z_j^{[L]}$, $\partial \hat{y}_k / \partial z_j^{[L]} = \hat{y}_k(\delta_{kj} - \hat{y}_j)$, y por la regla de la cadena:

$$dz_j^{[L]} = \sum_{k=1}^{n_y} \left(-\frac{y_k}{\hat{y}_k}\right) \hat{y}_k(\delta_{kj} - \hat{y}_j) = -y_j + \hat{y}_j \sum_{k=1}^{n_y} y_k = \hat{y}_j - y_j.$$

Paso 1: gradiente en la capa L

$$\begin{aligned} dz^{[L]} &= \hat{y} - y \in \mathbb{R}^{n_y}, \\ dW^{[L]} &= dz^{[L]}\, a^{[L-1]\top} \in \mathbb{R}^{n_y \times n^{[L-1]}}, \\ db^{[L]} &= dz^{[L]} \in \mathbb{R}^{n_y}. \end{aligned}$$

Retropropagación con softmax: hacia las capas ocultas

Paso 2: propagar el error a la capa $L-1$

$$da^{[L-1]} \equiv \frac{\partial \mathcal{L}}{\partial a^{[L-1]}} = W^{[L]\top} dz^{[L]},$$

$$da_i^{[L-1]} = \sum_{k=1}^{n_y} w_{ki}^{[L]} (\hat{y}_k - y_k) \quad \text{(componente } i\text{)},$$

$$dz^{[L-1]} = da^{[L-1]} \odot g^{[L-1]\prime}(z^{[L-1]}),$$

$$dW^{[L-1]} = dz^{[L-1]}\, a^{[L-2]\top}, \qquad db^{[L-1]} = dz^{[L-1]}.$$

- Cada neurona oculta recibe los errores de las n_y clases, ponderados por sus pesos hacia cada salida.
- **Paso 3:** repetir para $l = L-2, \ldots, 1$ con la regla de las capas ocultas.

Ejemplo numérico: propagación hacia adelante

- **Una** capa oculta ($l = 1$) con dos neuronas ReLU, $a_1^{[1]}$ y $a_2^{[1]}$, y salida softmax ($L = 2$, $n_y = 3$); sin sesgos.
- Un ejemplo $x = (1;\ 2)^\top$ de la clase 2: $y = (0;\ 1;\ 0)^\top$.

$$W^{[1]} = \begin{pmatrix} 1 & -1 \\ 0{,}5 & 0{,}5 \end{pmatrix} \in \mathbb{R}^{2\times 2}, \qquad W^{[2]} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \in \mathbb{R}^{3\times 2}.$$

Hacia adelante

$$z^{[1]} = W^{[1]}x = (-1;\ 1{,}5)^\top, \quad a^{[1]} = \mathrm{ReLU}(z^{[1]}) = (0;\ 1{,}5)^\top,$$
$$z^{[2]} = W^{[2]}a^{[1]} = (0;\ 1{,}5;\ 1{,}5)^\top, \quad \mathcal{L} = -\ln \hat{y}_2 = 0{,}799,$$
$$\hat{y} = a^{[2]} = \mathrm{softmax}(z^{[2]}) = (0{,}100;\ 0{,}450;\ 0{,}450)^\top.$$

Ejemplo numérico: retropropagación

Hacia atrás: capa de salida ($L = 2$) y luego capa oculta ($l = 1$)

$$dz^{[2]} = \hat{y} - y = (0{,}100;\ -0{,}550;\ 0{,}450)^\top,$$

$$dW^{[2]\top} = a^{[1]} dz^{[2]\top} = \begin{pmatrix} 0 & 0 & 0 \\ 0{,}151 & -0{,}825 & 0{,}675 \end{pmatrix},$$

$$da^{[1]} = W^{[2]\top} dz^{[2]} = (0{,}550;\ -0{,}100)^\top,\ \mathrm{ReLU}'(z^{[1]}) = (0;\ 1)^\top,$$

$$dz^{[1]} = da^{[1]} \odot \mathrm{ReLU}'(z^{[1]}) = (0;\ -0{,}100)^\top,$$

$$dW^{[1]} = dz^{[1]} x^\top = \begin{pmatrix} 0 & 0 \\ -0{,}100 & -0{,}201 \end{pmatrix}.$$

La neurona oculta 1 está apagada ($z_1^{[1]} < 0$): sus pesos no se actualizan.

Referencias

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, New York.

Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer, Cham.

Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.